Project Report

Project Title: Keylogger with Encrypted Data Exfiltration

Submitted By: Diwakar

Internship: Elevate Labs

Date: October 29, 2025

Abstract

This report details the development of a proof-of-concept ethical hacking tool, "Keylogger with Encrypted Data Exfiltration," as part of an internship project at Elevate Labs. The project consists of two core components: a stealthy keylogger client (enhanced_keylogger.py) and a powerful command-and-control server (enhanced_server.py). The keylogger is designed for Windows environments, featuring persistence, stealth, and strong AES encryption of captured data. The server, built with Flask, provides a dynamic web-based control panel for real-time monitoring, decryption, and remote management of the client. The entire system is designed for educational purposes to demonstrate the mechanics of data exfiltration and the importance of defensive cybersecurity measures.

---

1. Introduction

1.1. Project Background

In the field of cybersecurity, understanding offensive tactics is crucial for building robust defensive strategies. Keyloggers are a classic example of spyware used to illicitly capture keystrokes from a compromised system. This project was undertaken to deconstruct the functionality of such tools in a controlled, educational environment.

1.2. Project Objective

The primary objective was to build a proof-of-concept keylogger that securely captures and exfiltrates data. The key requirements were:

- **Stealth:** The keylogger should operate discreetly on the target machine.
- **Persistence:** It should automatically run on system startup.

- **Encryption:** All captured data must be encrypted before transmission to prevent inspection.
- **Exfiltration:** Data must be sent to a remote server.
- **Command & Control (C2):** The server should provide a user interface to view data and manage the client.

1.3. Ethical Disclaimer

This project and its components were developed strictly for educational and research purposes. The tool should only be used on systems for which explicit, written permission has been granted. Unauthorized deployment of this software is illegal and unethical. Elevate Labs and the developer are not responsible for any misuse.

---

2. System Architecture

The project operates on a client-server model.

1. **The Client (enhanced_keylogger.py):** This script runs on the target (victim) machine. Its responsibilities include capturing keystrokes, establishing persistence, encrypting the data, and sending it to the server.
2. **The Server (enhanced_server.py):** This script runs on the attacker's machine (in this case, a phone via Termux). It listens for incoming connections, receives and decrypts the data, stores it, and hosts a web-based control panel for monitoring.

The data flow is as follows: **Client-Side:** Key Generation → Initial Key Transfer to Server → Keystroke Capture → Data Encryption → Periodic POST Request to Server

Server-Side: Receive Key → Listen for Data → Receive Encrypted Data → Decrypt Data → Display in Web UI & Save to Log File

---

3. Technologies Used

- **Core Language:** Python 3
- **Web Framework:** Flask (for the server and control panel)
- **Keystroke Monitoring:** pynput
- **Encryption:** cryptography (specifically Fernet symmetric encryption)

- **HTTP Communication:** requests
- **Windows Integration:** winreg, ctypes (for persistence and stealth)
- **Standard Libraries:** os, sys, threading, json, base64, subprocess

---

4. Implementation Details: The Keylogger Client

The client script is engineered for stealth, resilience, and autonomous operation.

4.1. Stealth and Persistence

To avoid easy detection, the keylogger implements several stealth mechanisms:

1. **Self-Replication:** On its first run, the copy_and_execute() function copies the script to a less suspicious, hidden directory: $C:\Windows\SystemTemp$. It then executes this new copy and the original script terminates.
2. **Hidden Attributes:** Both the new folder (SystemTemp) and the script within it have their file attributes set to **Hidden** using the Windows API (ctypes.windll.kernel32.SetFileAttributesW). This prevents them from appearing in a standard File Explorer view.
3. **Startup Persistence:** The add_to_startup() function adds an entry to the Windows Registry under HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run. This ensures the keylogger is automatically executed every time the user logs in.

4.2. Encryption and Key Management

Security of the captured data is paramount.

1. **Key Generation:** A unique, cryptographically strong encryption key is generated using Fernet.generate_key() on the client's first run. This key is saved to a hidden file (decryption_key.key) in the script's directory.
2. **Key Exfiltration:** The send_encryption_key() function immediately sends this key, encoded in Base64, to the server's /send_key endpoint. This happens only once.
3. **Data Encryption:** All subsequent keystroke data captured is encrypted using the generated Fernet key before being sent over the network.

4.3. Data Capture and Exfiltration

1. **Keystroke Logging:** The pynput.keyboard.Listener runs in the background, capturing every key press. Special keys like Enter, Space, and Backspace are handled to format the logs correctly.
2. **Periodic Transmission:** Instead of sending every keystroke individually (which would be noisy), the send_post_req() function collects them in a buffer. A threading.Timer sends the collected data to the server at a regular interval (default: 10 seconds).
3. **Adaptive Retry Logic:** If the server is unreachable, the client doesn't stop. It retries 10 times at the normal interval. If it still fails, it switches to a much slower interval (120 seconds) to reduce network traffic and avoid raising suspicion, before attempting again.

---

5. Implementation Details: The C2 Server

The server is the brain of the operation, providing a central point for data collection and management.

5.1. Server and API Endpoints

The Flask server is lightweight and efficient, defining several key API routes:

- GET /: Serves the main HTML/CSS/JavaScript control panel.
- POST /send_key: A dedicated endpoint to receive the initial encryption key from a new client.
- POST /: The main endpoint for receiving encrypted keystroke data.
- GET /control/status: Provides a JSON summary of the server's status.
- POST /control/...: A set of endpoints (kill, enable_startup, etc.) for remote control actions.

5.2. Data Handling and Decryption

When encrypted data arrives at the POST / endpoint, the server performs the following actions:

1. Writes the raw, encrypted data to a log file named with the current date (e.g., 2025-10-29_keyboard_capture.txt).
2. Uses the previously received Fernet key to decrypt the data.

3. If decryption is successful, it writes the plaintext keystrokes to a separate, decrypted log file (e.g., 2025-10-29_keyboard_capture_decrypted.txt).
4. Adds the decrypted log to an in-memory buffer to be displayed on the web interface in real-time.

5.3. The Web Control Panel

The server's most powerful feature is its futuristic, hacker-themed web interface. It provides:

- **Live Status Indicator:** A visual indicator shows the client's status:
    - **WAITING:** The server is running but has not yet received a key from any client.
    - **ONLINE:** The client is actively sending data.
    - **OFFLINE:** The server has not received data for over 5 minutes, suggesting the client machine is off or has lost connection.
- **Live Log Feed:** A terminal-like window that displays the latest decrypted keystrokes as they arrive, automatically refreshing every 5 seconds.
- **Remote Control Panel:** Buttons to perform actions like sending a kill signal to the client.

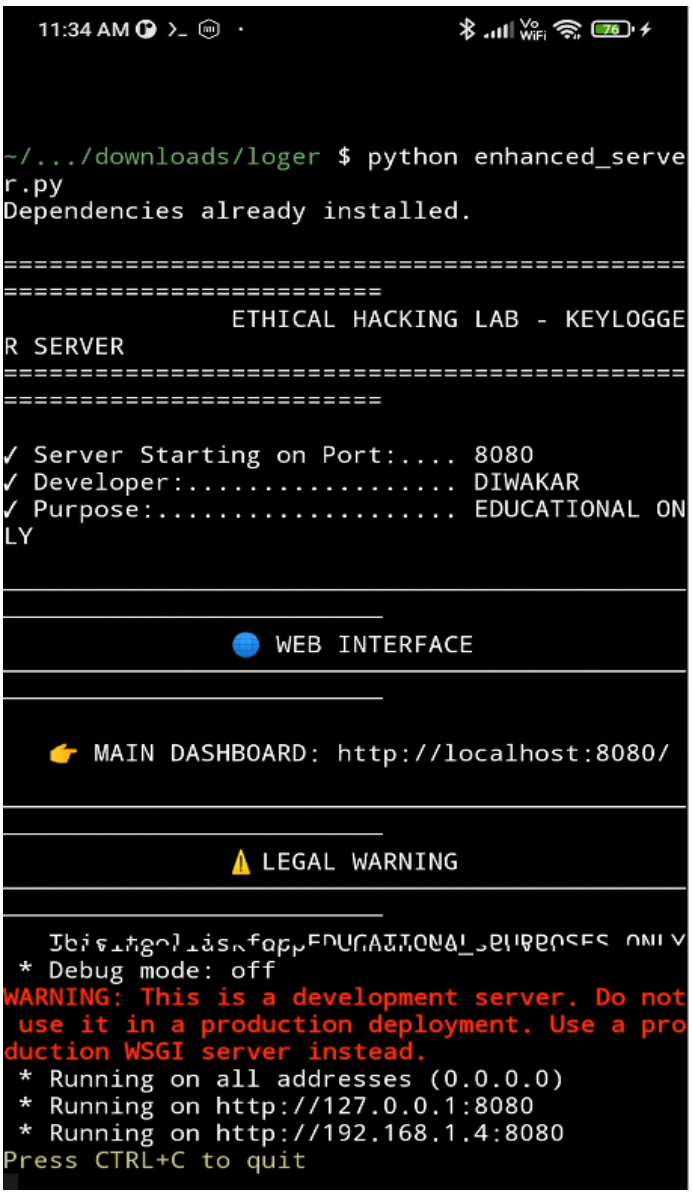
---

6. Practical Walkthrough

This section outlines the steps to deploy and test the project, using a computer as the victim and a phone with Termux as the server.

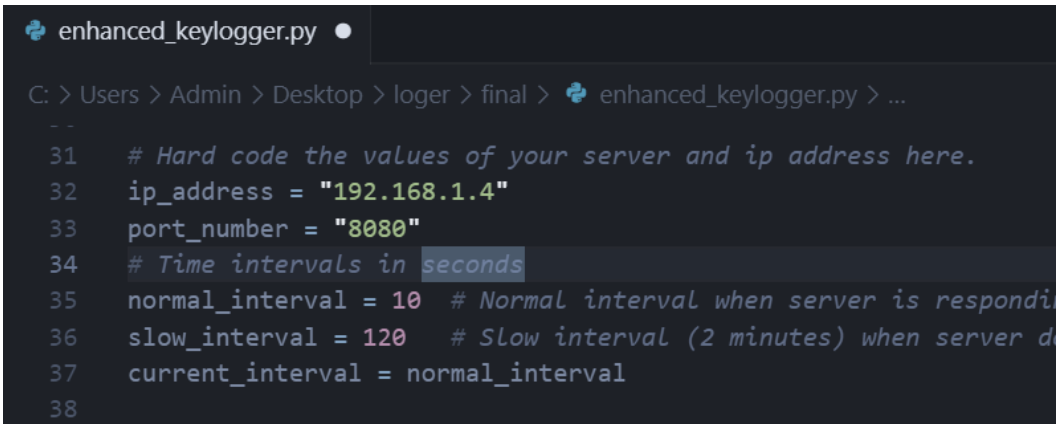
Step 1: Setup the Server on Termux First, start the Flask server on your phone. It will listen for connections from your local network.

- **Command:**
  pkg update && pkg upgrade -y
  pkg install rust clang python-pip -y
  pip install –upgrade pip setuptools wheel
  python enhanced_server.py
- **Expected Output:** The terminal will display the server's status and the URL to access the control panel (e.g., http://192.168.1.4:8080).



Open a browser and navigate to this URL. You will see the control panel with the status as **WAITING**.

Step 2: Configure and Run the Keylogger on the Computer Open the enhanced_keylogger.py script and edit the ip_address variable to match the IP address of your phone (shown in the Termux output).

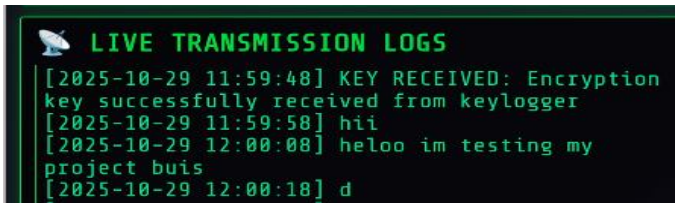


Now, run the keylogger script on your Windows computer. The script will appear to do nothing, but it has already copied itself to $C:\Windows\SystemTemp$ and started running from there.

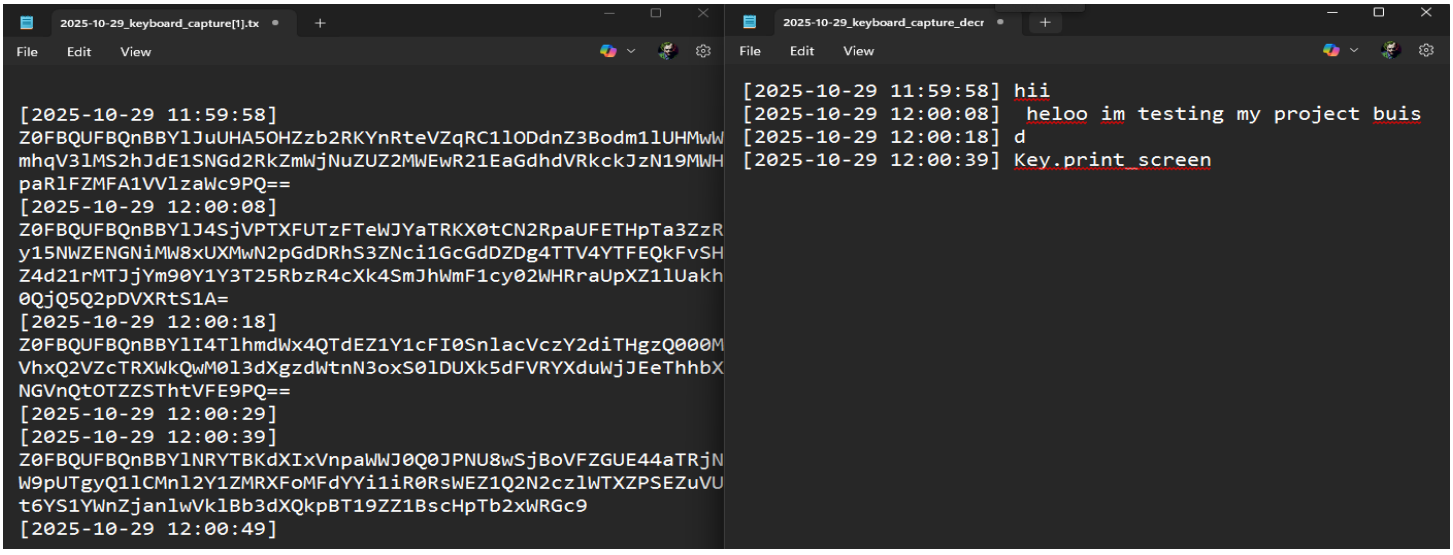
Step 3: Monitoring the Connection Check the server's web interface. Within seconds, two things will happen:

1. A log entry will appear: [YYYY-MM-DD HH:MM:SS] KEY RECEIVED...
2. The status indicator will change from WAITING to **ONLINE**.

Step 4: Capturing Keystrokes On the computer, type any text in any application (e.g., Notepad, a web browser). Wait for the 10-second interval to pass. The text you typed will appear in the "Live Transmission Logs" on the server's web panel.



Step 5: Checking the Log Files On the machine running the server (your phone), check the directory where the script is located. You will find a new folder received_logs containing the encrypted and decrypted text files, organized by date.



Future Enhancements

While this project is a comprehensive proof-of-concept, it can be extended with more advanced features to better simulate real-world threats. This understanding helps in developing more robust defensive measures.

- **HTTPS Integration:** The current communication is over HTTP. Implementing TLS/SSL to serve the panel over HTTPS would encrypt the entire communication channel, not just the data payload, protecting against man-in-the-middle (MITM) attacks.
- **Code Obfuscation and Evasion:** The keylogger is a Python script, which is easy to analyze. Compiling the script into a standalone Windows executable (.exe) using tools like PyInstaller would make it harder to reverse-engineer. Further obfuscation techniques could be applied to evade detection by antivirus (AV) software.
- **Feature Extensions:** The keylogger's capabilities could be expanded beyond keystroke logging. Additional modules could be developed for:
    - Screenshot capture on a timer or trigger.
    - Microphone audio recording.
    - Exfiltration of specific files from the target system.
- **Stealthier C2 Channels:** Using standard HTTP POST requests for command and control is effective but can be detected by network monitoring. More advanced malware often uses covert channels, such as hiding C2 traffic within DNS queries (DNS tunneling) or using public services like social media platforms or cloud storage APIs for communication.

---

8. Conclusion

This project successfully met all its objectives, resulting in a functional proof-of-concept keylogger with a sophisticated command-and-control server. The implementation demonstrates key concepts in ethical hacking, including system persistence, stealth techniques, secure data transmission using encryption, and remote client management.

The hands-on experience gained at Elevate Labs through this project has been invaluable, providing deep insights into the offensive security mechanisms that defenders must protect against. It serves as a powerful educational tool for understanding and mitigating real-world cyber threats.